

## 23CSE202 DATABASE MANAGEMENT SYSTEMS

### S3 B.Tech CSE

Department of Computer Science and Engineering

Amrita School of Computing, Amritapuri Campus

### 3. Database Design Document

Group No : C13

Team Members :

| Roll No          | Student Name      |
|------------------|-------------------|
| AM.SC.U4CSE25202 | ANAGHA SD         |
| AM.SC.U4CSE25203 | ANANTH KRISHNA CS |
| AM.SC.U4CSE25214 | DEVINANDANA P     |
| AM.SC.U4CSE25242 | PRANAV V N G      |

Project Title : Campus Canteen Pre-Order and Queue Management System

#### 3.1. Detailed 1. Overview of the System

The **Campus Canteen Pre-Order and Queue Management System** is designed to reduce the waiting time and overcrowding that usually happens in a college canteen during busy hours. In the existing situation, students have to stand in a queue to place their food orders and may not know how long it will take to receive their food. The proposed system allows students to **browse the daily menu, place pre-orders, select a pickup slot, and track their orders** without having to wait at the counter.

The system mainly consists of three types of users: **Students, Canteen Staff, and Admin**. Each user has different responsibilities and access to the system. Students are mainly involved in ordering and tracking food, staff members handle the preparation and processing of orders, and the admin manages the menu, pricing, stock, and reports.

### Main Entities and Attributes

The important entities in the system are **Students, Staffs, MenuItem, Orders, OrderItems, OrderStatusLog, and KitchenCapacity**.

The **Student** entity stores information about students who use the system. The student can place orders and view their own order history.

The **Staff** entity stores information about staff members who use the system. The staffs can place orders and view their own order history.

The **MenuItem** entity contains details about the food available in the canteen. Its important attributes include **item name, category, price, preparation time, and daily stock**. The availability of a menu item depends on its current stock.

The **Order** entity represents an order placed by a student. It contains information related to the order and its current status. An order can contain one or more food items, which are represented using the **OrderItems** entity.

The **OrderStatusLog** entity is used to maintain the history of an order's status changes. Whenever an order changes from one stage to another, the system records the new status and the corresponding timestamp.

The **KitchenCapacity** entity stores information about the current kitchen workload and capacity. This information is used while calculating the estimated waiting time for new orders. These entities are planned as separate, well-normalized tables in the PostgreSQL database.

## Relationships Between Entities

A **student can place multiple orders**, while each order belongs to a particular student. An **order can contain multiple menu items**, and a menu item can be included in different orders. The **OrderItems** entity is used to represent the items included in each order.

Each order also has a sequence of status changes. The **OrderStatusLog** keeps track of these changes, such as when the order was placed, when preparation started, when it became ready, and when it was collected.

KitchenCapacity is a standalone configuration table used by the waiting-time calculation.

## Users of the System

### Student

Students can log in to the system, view the available menu, check prices and stock, select food items, and place a pre-order for a pickup slot. After placing an order, they can track its status and view their previous orders. Students are restricted to viewing their own orders.

### Canteen Staff

Canteen staff handle the incoming orders through a **live queue dashboard**. They can view active orders and update their status as the food is being prepared. They move orders through **Placed → Preparing → Ready → Collected**. Staff can also monitor the kitchen capacity to manage the workload efficiently.

### Admin

The admin manages the overall canteen information. The admin can add or edit menu items, update prices and daily stock, and monitor system information. The admin can also view reports such as **daily sales, popular food items, peak ordering hours, and average preparation time**.

## Business Rules

The system follows several rules to keep the ordering process organized and maintain correct data.

**1) Only available menu items can be ordered.** When the stock of an item reaches zero, the item is automatically marked as unavailable or sold out.

**2) Orders follow a fixed status flow:**

**Placed → Preparing → Ready → Collected.** Every status change is recorded with its timestamp in the OrderStatusLog table using a database trigger.

**3) Pre-orders are prioritized** based on their booking time or selected pickup slot so that they can be processed properly along with walk-in orders.

**4) Waiting time is estimated** using the number of pending orders, preparation time of food items, and current kitchen capacity.

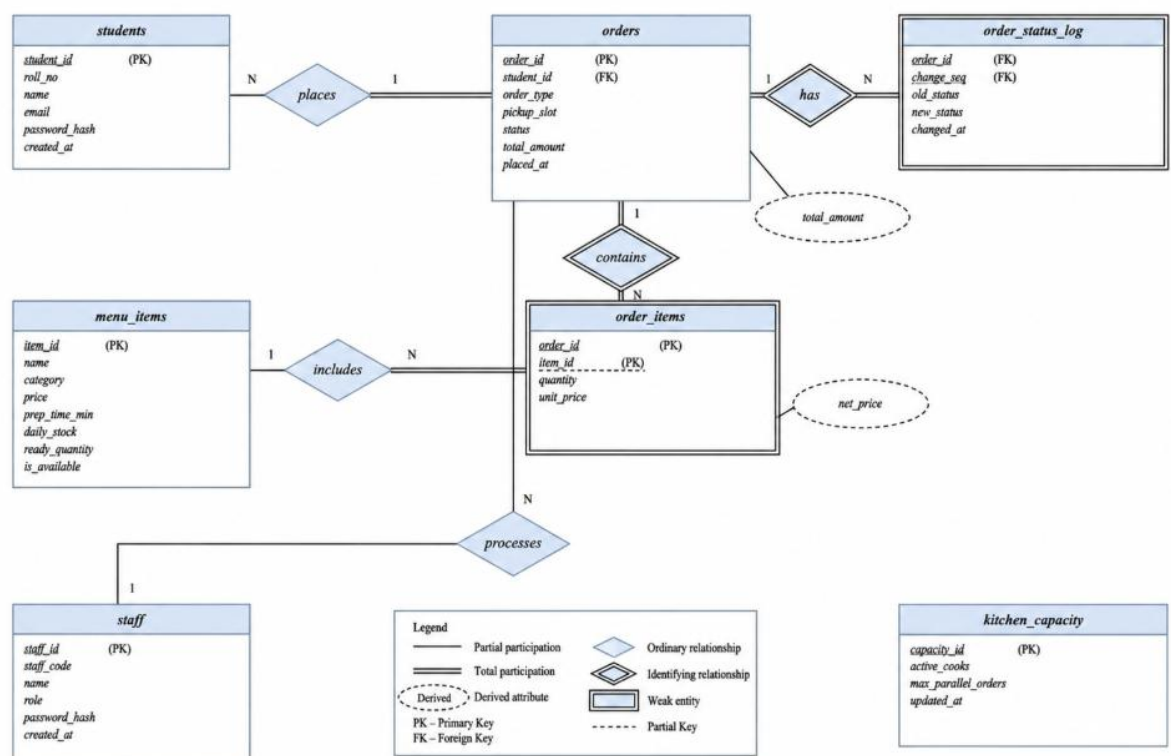
**5) Different users have different permissions.** Students can access their own orders, staff can manage active orders, and admins can manage menu and system-wide information.

**6) Order placement must maintain data consistency.** If an item becomes unavailable while an order is being placed, the complete transaction is rolled back instead of saving an incomplete order.

Overall, the system connects **students, food items, orders, kitchen capacity, staff processing, and administrative management** in one database-based application. This makes the canteen ordering process more organized and reduces unnecessary waiting and confusion during peak hours.

description of the project

### 3.2. ER Diagram



### 3.3 Relational Schema

#### 1.students (Relation)

| students          |                   |                           |
|-------------------|-------------------|---------------------------|
| <u>student_id</u> | INT / VARCHAR(20) | PRIMARY KEY               |
| roll_no           | VARCHAR(20)       | UNIQUE, NOT NULL          |
| name              | VARCHAR(100)      | NOT NULL                  |
| email             | VARCHAR(100)      | UNIQUE, NOT NULL          |
| password_hash     | VARCHAR(255)      | NOT NULL (                |
| created_at        | TIMESTAMP         | DEFAULT CURRENT_TIMESTAMP |

students(student\_id, roll\_no, name, email, password\_hash, created\_at) Integrity Notes: Represents campus student accounts. roll\_no and email maintain candidate key uniqueness constraints.

#### 2.staff (Relation)

| staff           |                   |                           |
|-----------------|-------------------|---------------------------|
| <u>staff_id</u> | INT / VARCHAR(20) | PRIMARY KEY               |
| staff_code      | VARCHAR(20)       | UNIQUE, NOT NULL          |
| name            | VARCHAR(100)      | NOT NULL                  |
| role            | VARCHAR(50)       | NOT NULL                  |
| password_hash   | VARCHAR(255)      | NOT NULL                  |
| created_at      | TIMESTAMP         | DEFAULT CURRENT_TIMESTAMP |

staff(staff\_id, staff\_code, name, role, password\_hash, created\_at) Integrity Notes: Standalone operational entity for canteen kitchen staff, counter operators, and supervisors.

### 3.menu\_items (Relation)

| menu_items     |                   |                              |
|----------------|-------------------|------------------------------|
| <u>item_id</u> | INT / VARCHAR(20) | PRIMARY KEY                  |
| name           | VARCHAR(100)      | NOT NULL                     |
| category       | VARCHAR(50)       | NOT NULL                     |
| price          | DECIMAL(10,2)     | NOT NULL, CHECK (price >= 0) |
| prep_time_min  | INT               | NOT NULL                     |
| daily_stock    | INT               | NOT NULL                     |
| ready_quantity | INT               | NOT NULL                     |
| is_available   | BOOLEAN           | DEFAULT TRUE                 |

menu\_items(item\_id, name, category, price, prep\_time\_min, daily\_stock, ready\_quantity, is\_available)

### 4.orders (Relation)

| orders          |                   |                                             |
|-----------------|-------------------|---------------------------------------------|
| <u>order_id</u> | INT / VARCHAR(20) | PRIMARY KEY                                 |
| student_id      | INT / VARCHAR(20) | FOREIGN KEY REFERENCES students(student_id) |
| order_type      | VARCHAR(20)       | NOT NULL                                    |
| pickup_slot     | VARCHAR(30)       | NOT NULL                                    |
| status          | VARCHAR(20)       | NOT NULL                                    |
| total_amount    | DECIMAL(10,2)     | DERIVED ATTRIBUTE                           |
| placed_at       | TIMESTAMP         | DEFAULT CURRENT_TIMESTAMP                   |

orders(order\_id, student\_id, order\_type, pickup\_slot, status, total\_amount, placed\_at)

Integrity Notes: Foreign Key student\_id maps the 1:N 'places' relationship.

total\_amount is derived from o

## 5.order\_items (Relation)

| order_items     |                   |                                                                               |
|-----------------|-------------------|-------------------------------------------------------------------------------|
| <u>order_id</u> | INT / VARCHAR(20) | <b>PRIMARY KEY, FOREIGN KEY</b> REFERENCES orders(order_id) ON DELETE CASCADE |
| <u>item_id</u>  | INT / VARCHAR(20) | <b>PRIMARY KEY, FOREIGN KEY</b> REFERENCES menu_items(item_id)                |
| quantity        | INT               | NOT NULL, CHECK (quantity > 0)                                                |
| unit_price      | DECIMAL(10,2)     | NOT NULL                                                                      |
| net_price       | DECIMAL(10,2)     | DERIVED ATTRIBUTE (quantity * unit_price)                                     |

order\_items(order\_id, item\_id, quantity, unit\_price) Integrity Notes: Weak entity resolving the M:N relationship between orders and menu\_items. Composite Primary Key (order\_id, item\_id). Net\_price is computed dynamically.

## 6.order\_status\_log (Relation)

| order_status_log  |                   |                                                                               |
|-------------------|-------------------|-------------------------------------------------------------------------------|
| <u>order_id</u>   | INT / VARCHAR(20) | <b>PRIMARY KEY, FOREIGN KEY</b> REFERENCES orders(order_id) ON DELETE CASCADE |
| <u>change_seq</u> | INT               | <b>PRIMARY KEY</b> Partial Key / Sequence number                              |
| old_status        | VARCHAR(20)       | NULL on initial order creation                                                |
| new_status        | VARCHAR(20)       | NOT NULL                                                                      |
| changed_at        | TIMESTAMP         | DEFAULT CURRENT_TIMESTAMP                                                     |

order\_status\_log(order\_id, change\_seq, old\_status, new\_status, changed\_at) Integrity Notes: Weak entity identified by orders via the 'has' identifying relationship. Composite Primary Key (order\_id, change\_seq) guarantees sequential audit tracking per order.

## 7.kitchen\_capacity (Relation)

| kitchen_capacity    |                   |                                            |
|---------------------|-------------------|--------------------------------------------|
| <b>capacity_id</b>  | INT / VARCHAR(20) | <b>PRIMARY KEY</b>                         |
| active_cooks        | INT               | NOT NULL, CHECK (active_cooks >= 0)        |
| max_parallel_orders | INT               | NOT NULL, CHECK (max_parallel_orders >= 0) |
| updated_at          | TIMESTAMP         | DEFAULT CURRENT_TIMESTAMP ON UPDATE        |

kitchen\_capacity(capacity\_id, active\_cooks, max\_parallel\_orders, updated\_at)  
Integrity Notes: Configuration table managing kitchen workflow concurrency limits based on on-duty staffing.

| students          |              |                           |
|-------------------|--------------|---------------------------|
| Attribute         | Data Type    | Constraints               |
| <b>student_id</b> | INT          | PRIMARY KEY               |
| roll_no           | VARCHAR(20)  | UNIQUE, NOT NULL          |
| name              | VARCHAR(100) | NOT NULL                  |
| email             | VARCHAR(100) | UNIQUE, NOT NULL          |
| password_hash     | VARCHAR(255) | NOT NULL                  |
| created_at        | TIMESTAMP    | DEFAULT CURRENT_TIMESTAMP |

| staff           |              |                           |
|-----------------|--------------|---------------------------|
| Attribute       | Data Type    | Constraints               |
| <b>staff_id</b> | INT          | PRIMARY KEY               |
| staff_code      | VARCHAR(20)  | UNIQUE, NOT NULL          |
| name            | VARCHAR(100) | NOT NULL                  |
| role            | VARCHAR(50)  | NOT NULL                  |
| password_hash   | VARCHAR(255) | NOT NULL                  |
| created_at      | TIMESTAMP    | DEFAULT CURRENT_TIMESTAMP |

| menu_items     |               |                       |
|----------------|---------------|-----------------------|
| Attribute      | Data Type     | Constraints           |
| <b>item_id</b> | INT           | PRIMARY KEY           |
| name           | VARCHAR(100)  | NOT NULL              |
| category       | VARCHAR(50)   | NOT NULL              |
| price          | DECIMAL(10,2) | NOT NULL (price >= 0) |
| prep_time_min  | INT           | NOT NULL              |
| daily_stock    | INT           | NOT NULL              |
| ready_quantity | INT           | NOT NULL              |
| is_available   | BOOLEAN       | DEFAULT TRUE          |

| orders          |               |                                             |
|-----------------|---------------|---------------------------------------------|
| Attribute       | Data Type     | Constraints                                 |
| <b>order_id</b> | INT           | PRIMARY KEY                                 |
| student_id      | INT           | FOREIGN KEY REFERENCES students(student_id) |
| order_type      | VARCHAR(20)   | NOT NULL                                    |
| pickup_slot     | VARCHAR(50)   | NOT NULL                                    |
| status          | VARCHAR(20)   | NOT NULL                                    |
| total_amount    | DECIMAL(10,2) | DERIVED ATTRIBUTE                           |
| placed_at       | TIMESTAMP     | DEFAULT CURRENT_TIMESTAMP                   |

| order_items     |               |                                                         |
|-----------------|---------------|---------------------------------------------------------|
| Attribute       | Data Type     | Constraints                                             |
| <b>order_id</b> | INT           | PRIMARY KEY, FOREIGN KEY REFERENCES orders(order_id)    |
| <b>item_id</b>  | INT           | PRIMARY KEY, FOREIGN KEY REFERENCES menu_items(item_id) |
| quantity        | INT           | NOT NULL                                                |
| unit_price      | DECIMAL(10,2) | NOT NULL                                                |

| order_status_log  |             |                                                      |
|-------------------|-------------|------------------------------------------------------|
| Attribute         | Data Type   | Constraints                                          |
| <b>order_id</b>   | INT         | PRIMARY KEY, FOREIGN KEY REFERENCES orders(order_id) |
| <b>change_seq</b> | INT         | PRIMARY KEY                                          |
| old_status        | VARCHAR(50) | NOT NULL                                             |
| new_status        | VARCHAR(50) | NOT NULL                                             |
| changed_at        | TIMESTAMP   | DEFAULT CURRENT_TIMESTAMP                            |

| kitchen_capacity    |           |                           |
|---------------------|-----------|---------------------------|
| Attribute           | Data Type | Constraints               |
| <b>capacity_id</b>  | INT       | PRIMARY KEY               |
| current_load        | INT       | NOT NULL                  |
| max_parallel_orders | INT       | NOT NULL                  |
| updated_at          | TIMESTAMP | DEFAULT CURRENT_TIMESTAMP |

### 3.4 Normalization and Functional dependency analysis

#### Functional Dependencies and their Significans:

- Students:**

student\_id → roll\_no, name, email, password\_hash, created\_at  
email → student\_id, roll\_no, name, password\_hash, created\_at (email is unique — a second determinant).

Students FD keep login data unambiguous.

- Staff:**

staff\_id → staff\_code, name, role, password\_hash, created\_at

Staff FD keep login data unambiguous.

- MenuItems:**

item\_id → name, category, price, prep\_time\_min, daily\_stock, ready\_quantity,  
is\_available

Note the internal FD daily\_stock → is\_available (the trigger sets is\_available = false  
whenever daily\_stock = 0)

MenuItems FDs are what let the wait-time procedure trust a single source of truth per  
item.

- **Orders:**

order\_id → student\_id, order\_type, pickup\_slot, status, total\_amount, placed\_at  
Orders FDs isolate one student's order history from another's.

- **OrderItems** (weak entity):

(order\_id, item\_id) → quantity, unit\_price, and separately quantity, unit\_price →  
net\_price (since net\_price is a GENERATED column, purely computed from the other  
two).

- **OrderStatusLog** (weak entity):

(order\_id, change\_seq) → old\_status, new\_status, changed\_at  
The two weak-entity FDs are what justify their composite keys (below).

- **KitchenCapacity:** capacity\_id → active\_cooks, max\_parallel\_orders, updated\_at  
KitchenCapacity's FD keeps kitchen throughput independent of any one order.

**Candidate Key Justification (attribute closure) — using OrderItems, your weak  
entity**

### 1. Students

**Attributes:** {student\_id, roll\_no, name, email, password\_hash, created\_at}

**FDs:** student\_id → roll\_no, name, email, password\_hash, created\_at  
email → student\_id, roll\_no, name, password\_hash, created\_at

roll\_no → email, name, password\_hash, created\_at

**Closure of {student\_id}:**

{student\_id}<sup>+</sup> = {student\_id, roll\_no, name, email, password\_hash, created\_at} = full  
set (via FD1)

→ superkey. No proper subset exists (single attribute) → **{student\_id} is a candidate  
key.**

**Closure of {email}:**

{email}<sup>+</sup> = {email, student\_id, roll\_no, name, password\_hash, created\_at} = full set  
(via FD2)

→ superkey. Single attribute, trivially minimal → **{email} is a second candidate key.**

**Closure of {roll\_no} = {roll\_no email, name, password\_hash, created\_at**

**}** a superkey. And a minimal super key. Therefore a candidate key.

→ Students has **two candidate keys**: {student\_id}, {email}, {roll\_no}. student\_id is chosen as PK; email and roll\_no becomes a UNIQUE constraint (alternate key).

## 2. Staff

Attributes: {staff\_id, staff\_code, name, role, password\_hash, created\_at}

FD: staff\_id → staff\_code, name, role, password\_hash, created\_at

Closure of {staff\_id}: {staff\_id}<sup>+</sup> = {staff\_id, staff\_code, name, role, password\_hash, created\_at} = full set (via FD)

→ superkey. No proper subset exists (single attribute) → {staff\_id} is a candidate key.

Closure of {staff\_code}: no FD has staff\_code as a determinant → {staff\_code}<sup>+</sup> = {staff\_code} only → not a superkey.

→ Staff has one candidate key: {staff\_id}. staff\_id is chosen as PK.

## 3. MenuItems

Attributes: {item\_id, name, category, price, prep\_time\_min, daily\_stock, ready\_quantity, is\_available}

FD: item\_id → name, category, price, prep\_time\_min, daily\_stock, ready\_quantity, is\_available

Closure of {item\_id}: {item\_id}<sup>+</sup> = {item\_id, name, category, price, prep\_time\_min, daily\_stock, ready\_quantity, is\_available} = full set (via FD)

→ superkey. No proper subset exists (single attribute) → {item\_id} is a candidate key.

No other attribute is asserted as a determinant → MenuItems has one candidate key: {item\_id}. item\_id is chosen as PK.

## 4. Orders

Attributes: {order\_id, student\_id, order\_type, pickup\_slot, status, total\_amount, placed\_at}

FD: order\_id → student\_id, order\_type, pickup\_slot, status, total\_amount, placed\_at

Closure of {order\_id}: {order\_id}<sup>+</sup> = {order\_id, student\_id, order\_type, pickup\_slot, status, total\_amount, placed\_at} = full set (via FD)

→ superkey. No proper subset exists (single attribute) → {order\_id} is a candidate key.

No other attribute is asserted as a determinant → Orders has one candidate key: {order\_id}. order\_id is chosen as PK.

## 5. OrderItems (weak entity)

Attributes: {order\_id, item\_id, quantity, unit\_price, net\_price}

FDs: (order\_id, item\_id) → quantity, unit\_price

quantity, unit\_price → net\_price

Closure of {order\_id, item\_id}:

{order\_id, item\_id}<sup>+</sup> = {order\_id, item\_id, quantity, unit\_price} = (via FD1)

{order\_id, item\_id}<sup>+</sup> = {order\_id, item\_id, quantity, unit\_price, net\_price} = full set (via FD2, now that quantity and unit\_price are in the closure)

→ superkey.

Closure of {order\_id}: {order\_id}<sup>+</sup> = {order\_id} only (order\_id alone repeats across many items in the same order) → not a superkey.

Closure of {item\_id}: {item\_id}<sup>+</sup> = {item\_id} only (item\_id alone repeats across many orders) → not a superkey.

→ Neither proper subset reaches the full set, so {order\_id, item\_id} is minimal → {order\_id, item\_id} is the candidate key. item\_id is the partial key; order\_id is the owner's identifying key — this is what makes OrderItems a weak entity.

#### 6. OrderStatusLog (weak entity)

Attributes: {order\_id, change\_seq, old\_status, new\_status, changed\_at}

FD: (order\_id, change\_seq) → old\_status, new\_status, changed\_at

Closure of {order\_id, change\_seq}:

{order\_id, change\_seq}<sup>+</sup> = {order\_id, change\_seq, old\_status, new\_status, changed\_at} = full set (via FD)

→ superkey.

Closure of {order\_id}: {order\_id}<sup>+</sup> = {order\_id} only (order\_id alone repeats across every status change for that order) → not a superkey.

Closure of {change\_seq}: {change\_seq}<sup>+</sup> = {change\_seq} only (change\_seq alone repeats across different orders) → not a superkey.

→ Neither proper subset reaches the full set, so {order\_id, change\_seq} is minimal → {order\_id, change\_seq} is the candidate key. change\_seq is the partial key; order\_id is the owner's identifying key — this is what makes OrderStatusLog a weak entity.

#### 7. KitchenCapacity

Attributes: {capacity\_id, active\_cooks, max\_parallel\_orders, updated\_at}

FD: capacity\_id → active\_cooks, max\_parallel\_orders, updated\_at

Closure of {capacity\_id}: {capacity\_id}<sup>+</sup> = {capacity\_id, active\_cooks, max\_parallel\_orders, updated\_at} = full set (via FD)

→ superkey. No proper subset exists (single attribute) → {capacity\_id} is a candidate key.

No other attribute is asserted as a determinant → {capacity\_id} is the only candidate key.

### Normalization Walkthrough (1NF → BCNF)

#### Unnormalized relation — OrderDetails:

(order\_id, student\_id, roll\_no, student\_name, email, pickup\_slot, order\_status, item\_id, item\_name, category, unit\_price, prep\_time\_min, quantity)

**Step 1 — 1NF:** every attribute holds one atomic value, one row per (order, item) pair, no repeating groups → **1NF satisfied.**

Candidate key here: closure of {order\_id, item\_id} reaches every attribute (same logic as above, extended through the joined student/item data) → **{order\_id, item\_id}** is the key of this flat relation.

**Step 2 — 2NF:** check for partial dependencies (attribute depending on only part of the key).

- student\_id, pickup\_slot, order\_status depend only on order\_id → **partial dependency, violates 2NF**

- item\_name, category, unit\_price, prep\_time\_min depend only on item\_id → **also a partial dependency**

Decompose:

- Orders(order\_id, student\_id, pickup\_slot, order\_status)
- MenuItems(item\_id, item\_name, category, unit\_price, prep\_time\_min)
- OrderItems(order\_id, item\_id, quantity)
- StudentInfo(student\_id, roll\_no, student\_name, email)

→ **2NF satisfied** — every non-key attribute now depends on the whole key of its table.

**Step 3 — 3NF:** check for transitive dependencies (non-key attribute depending on another non-key attribute).

- In StudentInfo, email determines student\_name too — but since email is itself a candidate key (not a plain non-key attribute), 3NF explicitly permits this exception → no violation.
- No other transitive dependency exists among Orders, MenuItems, or the base OrderItems (before net\_price is added).

→ **3NF satisfied** for all four tables.

**Step 4 — BCNF:** every determinant must be a superkey, with no exception for candidate-key determinants.

- Orders, MenuItems, OrderItems: only determinant in each is the (composite, for OrderItems) key itself → OK.
- StudentInfo: two determinants, student\_id and email — **both are candidate keys** → satisfies BCNF too.

→ **All four decomposed tables satisfy BCNF.**

**Two deliberate exceptions worth naming, so your evaluator sees you caught them yourself:**

1. **MenuItems.is\_available** — technically a transitive FD (item\_id → daily\_stock → is\_available), which is a real 3NF violation on paper. It's kept anyway because it's fully maintained by a trigger (trg\_stock\_availability), so it can never drift out of sync — a deliberate, disclosed denormalization for query performance, not an oversight.
2. **OrderItems.net\_price** — same shape of transitive dependency (key → {quantity, unit\_price} → net\_price), but it's a Postgres GENERATED ALWAYS AS ... STORED column, meaning it's not independently writable — it can't ever hold an inconsistent value the way a normal stored transitive attribute could. This sidesteps the actual anomaly 3NF is designed to prevent, even though it looks like a violation on paper.

**Staff, KitchenCapacity, OrderStatusLog:** each has a single key (composite, in OrderStatusLog's case) with no other determinant present, so they're in BCNF by construction — no decomposition needed.